Data Generation

First need to make sure in the correct directory for it to work (load into top directory of the model that is being run) ensure to use the double slash and '' to meet syntax requirements for windows

In [1]:
```
import os
os.chdir('C:\\Users\\User\\Desktop\\CARE_Working_Folder\\Care_training_4')
print(os.getcwd())
print(os.listdir())
```

```
C:\Users\User\Desktop\CARE_Working_Folder\Care_training_4
['.ipynb_checkpoints', 'Step_1_Patch_Generation.ipynb', 'Step_2_Model_Training.ipynb
', 'Step_3_Model_Application.ipynb', 'Training']
```

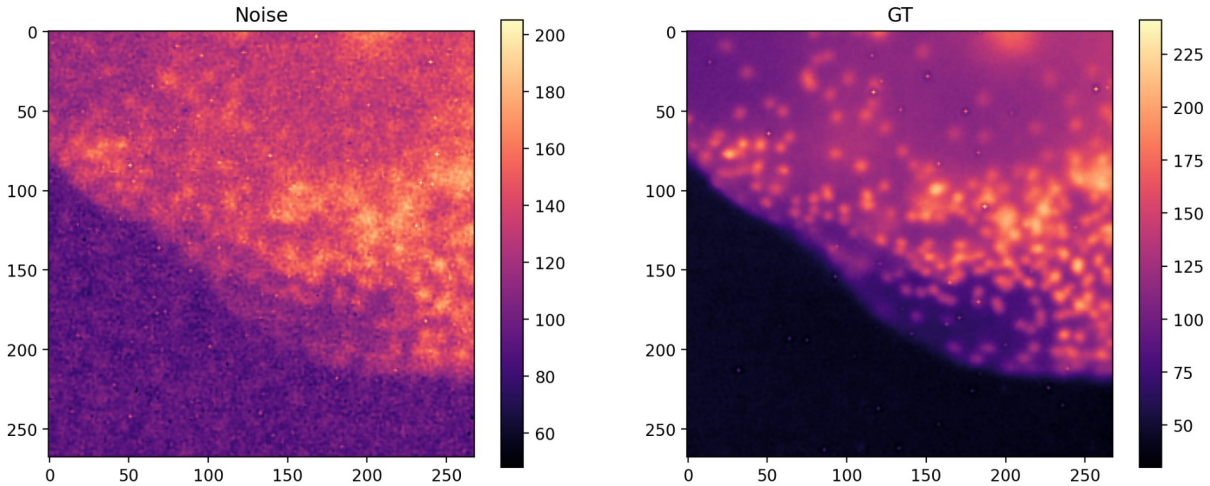
First is to import the appropraite packages needed for data generation

In [2]:
```
from __future__ import print_function, unicode_literals, absolute_import, division
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

from tifffile import imread
from csbdeep.utils import download_and_extract_zip_file, plot_some
from csbdeep.data import RawData, create_patches
```

Data needs to be split into testing and training in the 2d denoising example there is about 1:5 ratio of test to training data. below will display some of these images

In [3]:
```
y = imread('Training/GT_268/GT0015.tif')
x = imread('Training/NC_B_268/GT0015.tif')
print('image size =', x.shape)

plt.figure(figsize=(13,5))
plt.subplot(1,2,1)
plt.imshow(x, cmap  ="magma")
plt.colorbar()
plt.title("Noise")
plt.subplot(1,2,2)
plt.imshow(y, cmap  ="magma")
plt.colorbar()
plt.title("GT");
```

```
image size = (268, 268)
```



This is going to specify the raw data for the rest of the model, so it is what the patches will be generated from in the next steps

In [4]:
```
data = RawData.from_folder(basepath='Training', source_dirs=['N_268','N_B_268','N_M

n_images = data.size

for source_x, target_y, axes, mask in data.generator():

    pass
```

This code will generate the training patches from the variable 'data'

By default, patches are normalized based on a range of percentiles computed on the raw images, which tends to lead to more robust CARE networks in our experience. If not specified otherwise, patches which are purely background are also excluded from being extracted, since they do not contain interesting structures.

note: datta to be loaded in does not contain any background and therefore 'no background patches' should not effect patch generation

In [5]:
```
from csbdeep.data import no_background_patches, norm_percentiles, sample_percentile

X, Y, XY_axes = create_patches (
    raw_data            = data,
    patch_size          = (128,128),
    patch_filter        = no_background_patches(0),
    n_patches_per_image = 3,
    save_file           = 'FM4_training_data.npz',
)
```

```
=================================================================
 1680 raw images x    1 transformations   =  1680 images
 1680 images     x    3 patches per image =  5040 patches in total
=================================================================
Input data:
Training: target='GT_268', sources=['N_268', 'N_B_268', 'N_M_268', 'N_R_268', 'N_RMB
_268', 'NC_268', 'NC_B_268', 'NC_M_268', 'NC_R_268', 'NC_RMB_268'], axes='YX', patte
rn='*.tif'
=================================================================
Transformations:
1 x Identity
=================================================================
Patch size:
128 x 128
=================================================================
100%|██████████████████████████████████████████████████████████|
1680/1680 [00:11<00:00, 149.73it/s]
Saving data to FM4_training_data.npz.
```

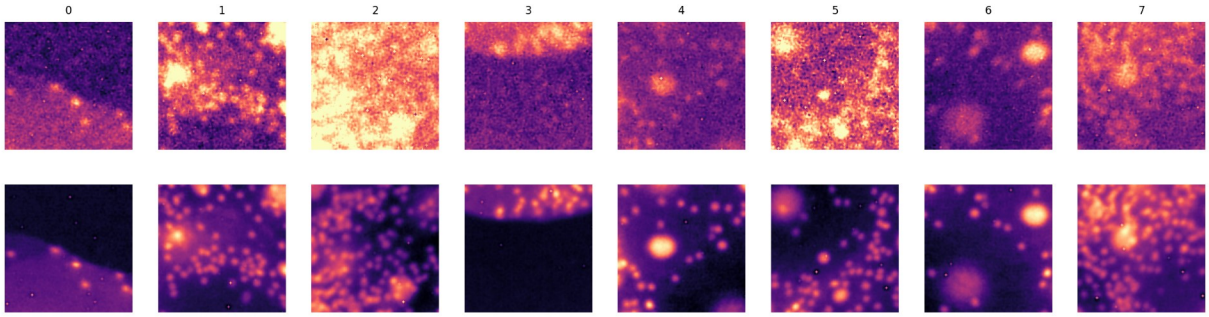
this will show us the dimensions of the patches how many, how many channels and the shape

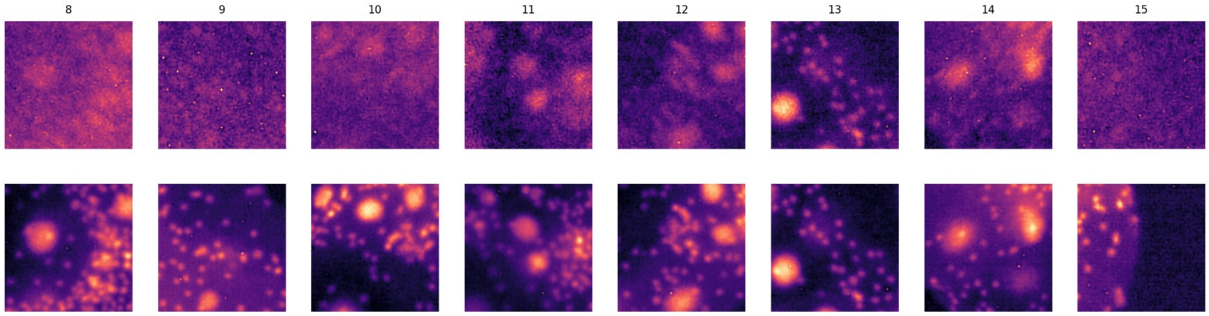
In [6]:
```
assert X.shape == Y.shape
print("shape of X,Y =", X.shape)
print("axes  of X,Y =", XY_axes)
```

```
shape of X,Y = (5040, 1, 128, 128)
axes  of X,Y = SCYX
```

next is to plot some example patches

In [7]:
```
for i in range(2):
    plt.figure(figsize=(16,4))
    sl = slice(8*i, 8*(i+1)), 0
    plot_some(X[sl],Y[sl],title_list=[np.arange(sl[0].start,sl[0].stop)])
    plt.show()
None;
```





In [ ]: